



JavaScript Interview Questions

What is typeof operator?

Entry

JavaScript provides a `typeof` operator that can examine a value and tell you what type it is:

```
var a;  
typeof a;           // "undefined"  
  
a = "hello world";  
typeof a;           // "string"  
  
a = 42;  
typeof a;           // "number"  
  
a = true;  
typeof a;           // "boolean"  
  
a = null;  
typeof a;           // "object" -- weird, bug  
  
a = undefined;  
typeof a;           // "undefined"  
  
a = { b: "c" };  
typeof a;           // "object"
```

What is equality in JavaScript ?

Entry

JavaScript has both strict and type-converting comparisons:

Strict comparison (e.g., `===`) checks for value equality without allowing *coercion*

Abstract comparison (e.g., `==`) checks for value equality with *coercion* allowed

```
var a = "42";  
var b = 42;  
  
a == b;      // true  
a === b;     // false
```

Some simple equality rules:

If either value (aka side) in comparison could be the true or false value, avoid == and use ===.

If either value in comparison could be of these specific values (0, "", or [] -- empty array), avoid == and use ===.

In all other cases, you're safe to use ==. Not only is it safe, but in many cases, it simplifies your code in a way that improves readability.

What is the object type?

Entry

The object type refers to a compound value where you can set properties (named locations) that each holds their own values of any type.

```
var obj = {  
  a: "hello world", // property  
  b: 42,  
  c: true  
};  
  
obj.a;      // "hello world", accessed with doted notation  
obj.b;      // 42  
obj.c;      // true  
  
obj["a"];   // "hello world", accessed with bracket notation  
obj["b"];   // 42  
obj["c"];   // true
```

Bracket notation is also useful if you want to access a property/key but the name is stored in another variable, such as:

```
var obj = {  
  a: "hello world",  
  b: 42  
};  
  
var b = "a";
```

```
obj[b];      // "hello world"
obj["b"];    // 42
```

Explain is Scope in JavaScript?

Entry

In JavaScript, each function gets its own *scope*. The scope is basically a collection of variables as well as the rules for how those variables are accessed by name. Only code inside that function can access that function's scoped variables.

A variable name has to be unique within the same scope. A scope can be nested inside another scope. If one scope is nested inside another, code inside the innermost scope can access variables from either scope.

Explain arrays in JavaScript

Entry

An array is an object that holds values (of any type) not particularly in named properties/keys, but rather in numerically indexed positions:

```
var arr = [
  "hello world",
  42,
  true
];

arr[0];      // "hello world"
arr[1];      // 42
arr[2];      // true
arr.length;  // 3

typeof arr;  // "object"
```

Explain what a callback function is and provide a simple example

Junior

A *callback* the function is a function that is passed to another function as an argument and is executed after some operation has been completed. Below is an example of a simple callback function that logs to the console *after* some operations have been completed.

```
function modifyArray(arr, callback) {
  // do something to arr here
```

```
arr.push(100);  
// then execute the callback function that was passed  
callback();  
}  
  
var arr = [1, 2, 3, 4, 5];  
  
modifyArray(arr, function() {  
  console.log("array has been modified", arr);  
});
```

Given a string, reverse each word in the sentence

Junior

```
var string = "Welcome to this Javascript Guide!";  
  
// Output becomes !ediuG tpircsavaJ siht ot emocleW  
var reverseEntireSentence = reverseBySeparator(string, "");  
  
// Output becomes emocleW ot siht tpircsavaJ !ediuG  
var reverseEachWord = reverseBySeparator(reverseEntireSentence, " ");  
  
function reverseBySeparator(string, separator) {  
  return string.split(separator).reverse().join(separator);  
}
```

Login to Unlock Full Access

Create ProTechStack account To Unlock 1000+ Expert Questions, Jobs & Much More



Continue with Google